

Veri Yapıları ve Manipülasyon

NTP II - Hafta 2



Bu sunum, "Veri Yapıları ve Manipölasyonu Temelleri" dersinin ikinci haftasında ele alınan konuları kapsamaktadır. Temel veri yapıları ve veri manipölasyonu teknikleri üzerine odaklanacağız.

Haftanın Hedefleri

Bu hafta sonunda aşağıdaki hedeflere ulaşmayı amaçlıyoruz:

- Birden fazla veriyi bir arada tutan koleksiyon tipi veri yapılarını (Liste, Demet, Sözlük, Küme) tanımak ve aralarındaki temel farkları açıklayabilmek.
- Listeler üzerinde eleman ekleme (append), silme (remove), güncelleme ve dilimleme (slicing) gibi temel manipülasyon işlemlerini yapabilmek.
- Sözlük (Dictionary) yapısını kullanarak, anahtar-değer (key-value) çiftleri ile yapılandırılmış verileri saklayabilmek ve anahtarlar üzerinden verilere hızlıca erişebilmek.

Haftanın Hedefleri

- String (metin) verileri üzerinde `.upper()`, `.lower()`, `.split()`, `.join()` gibi yaygın metotları kullanarak temel metin işleme görevlerini yerine getirebilmek.
- Her veri yapısının hangi senaryolarda daha avantajlı olduğu konusunda bilgi sahibi olmak.



Veri Yapıları Nedir ve Neden Önemlidir?

Veri yapıları, bilgisayarda verilerin düzenli ve verimli bir şekilde saklanması ve erişilmesini sağlayan özel formatlardır. Doğru veri yapısının seçilmesi, bir programın performansı üzerinde doğrudan etkilidir.

Örneğin, bir öğrencinin notlarını mı saklıyoruz, yoksa bir şehrin trafik akışını mı yönetiyoruz? Her senaryo için farklı veri yapıları daha uygun olabilir.



Koleksiyon Tipi Veri Yapılarına Genel Bakış

Python gibi modern programlama dillerinde, birden fazla veriyi bir arada tutan, yani koleksiyon tipi olarak adlandırılan çeşitli veri yapıları bulunur. Bu ders kapsamında dört temel koleksiyon tipini inceleyeceğiz:

- Liste (List)
- Demet (Tuple)
- Sözlük (Dictionary)
- Küme (Set)



Liste (List): Tanım ve Özellikler

Liste, Python'daki en sık kullanılan ve en esnek koleksiyon tipidir.

- Sıralıdır: Elemanlar belirli bir sıraya göre saklanır ve bu sıra korunur. İndeks numaraları (0'dan başlayarak) ile elemanlara erişilebilir.
- Değiştirilebilirdir (Mutable): Oluşturulduktan sonra elemanları eklenebilir, silinebilir veya güncellenebilir. Bu özelliği listeyi dinamik yapar.
- Farklı veri tiplerindeki elemanları bir arada tutabilir (örneğin, sayılar, metinler, hatta başka listeler).



Liste Oluşturma ve Elemanlara Erişim

Listeler köşeli parantezler `[]` arasına elemanlar yazılarak oluşturulur.

Kod Örneği:

```
my_list = [10, "Python", 20.5, True]
```

```
print(my_list) # Çıktı: 10
```

```
print(my_list[1]) # Çıktı: Python
```



Liste Manipülasyonu: `append()` ile Eleman Ekleme

Listelerin sonuna yeni bir eleman eklemek için `append()` metodu kullanılır. Bu, listenin sonuna yeni bir öge eklemenin en yaygın yoludur. Kod Örneği:

```
meyveler = ["elma", "muz"]  
meyveler.append("çilek")  
print(meyveler) # Çıktı: ['elma', 'muz', 'çilek']
```



append() metodunun aksine, insert(index, element) metodu ile liste içerisindeki istediğimiz bir indekse yeni bir eleman ekleyebiliriz. Mevcut elemanlar sağa doğru kaydırılır. Kod Örneği:

```
sayilar = [1, 2, 4, 5]
```

```
sayilar.insert(2, 3) # 2. indekse 3 ekle
```

```
print(sayilar) # Çıktı: [1-5]
```

Belirli bir değeri listeden silmek için remove() metodu kullanılır. Eğer listede birden fazla aynı değer varsa, sadece ilk bulunan eleman silinir. Kod Örneği:

```
urunler = ["kalem", "defter", "silgi", "kalem"]  
urunler.remove("kalem")  
print(urunler) # Çıktı: ['defter', 'silgi', 'kalem']
```



pop(index) metodu, belirli bir indeksteki elemanı listeden siler ve sildiği elemanı geri döndürür. Eğer indeks belirtilmezse, listenin sonundaki elemanı siler. Kod Örneği:

```
renkler = ["kırmızı", "mavi", "yeşil", "sarı"]  
silinen_renk = renkler.pop(1) # 1. indeksteki 'mavi' silinir  
print(renkler) # Çıktı: ['kırmızı', 'yeşil', 'sarı']  
print(silinen_renk) # Çıktı: mavi
```

Listeler değiştirilebilir (mutable) olduğu için, belirli bir indeksteki elemanın değerini yeni bir değerle kolayca güncelleyebiliriz. Kod Örneği:

```
notlar =
```

```
notlar[2] = 88 # 2. indeksteki elemanı güncelle
```

```
print(notlar)
```

Listenin belirli bir bölümünü (bir alt listesini) almak için dilimleme (slicing) kullanılır. Dilimleme [başlangıç:bitiş:adım] formatındadır. Bitiş indeksi dahil değildir. Kod Örneği:

```
alfabe = ['a', 'b', 'c', 'd', 'e', 'f']
```

```
alt_liste = alfabe[1:4] # 1. indeksten 4. indekse kadar (4. dahil değil)
```

```
print(alt_liste) # Çıktı: ['b', 'c', 'd']
```

Dilimleme sırasında adım (step) parametresi de kullanılabilir. Ayrıca negatif indeksler ve boş başlangıç/bitiş değerleri de mevcuttur. Kod Örneği:

```
rakamlar = [1-9]
```

```
print(rakamlar[::2]) # Çıktı: [2, 4, 6, 8] (2'şer atlayarak)
```

```
print(rakamlar[2:]) # Çıktı: [2-9] (2. indeksten sonuna kadar)
```

```
print(rakamlar[:-3]) # Çıktı: [1-6] (sondan 3. hariç)
```

Python, listelerle çalışırken faydalı olan bazı yerleşik fonksiyonlara sahiptir.

- `len(liste)` : Listenin eleman sayısını (uzunluğunu) verir.
- `max(liste)` : Listedeki en büyük değeri döndürür.
- `min(liste)` : Listedeki en küçük değeri döndürür. Kod Örneği:

```
sayilar = [5, 8, 10-12]
```

```
print(len(sayilar)) # Çıktı: 5
```

```
print(max(sayilar)) # Çıktı: 20
```

```
print(min(sayilar)) # Çıktı: 5
```



Bir alışveriş listesi oluşturma senaryosu, liste manipülasyonlarını göstermek için idealdir. `append()` ile yeni ürünler ekleyecek, `remove()` ile bir ürün çıkaracak ve belirli bir elemanı indeksi kullanılarak güncelleyeceğiz.

Uygulama Adımları:

1. Boş bir alışveriş listesi ile başla.
2. Birkaç ürün ekle.
3. Bir ürünü listeden çıkar.
4. Bir ürünün miktarını veya tanımını güncelle.

```
alisveris_listesi = []  
alisveris_listesi.append("Elma")  
alisveris_listesi.append("Süt")  
alisveris_listesi.append("Ekmek")  
print("Mevcut alışveriş listesi:", alisveris_listesi)  
# Çıktı: Mevcut alışveriş listesi: ['Elma', 'Süt', 'Ekmek']
```

```
alisveris_listesi = ['Elma', 'Süt', 'Ekmek', 'Yoğurt'] # Önceki adımdan devam
alisveris_listesi.remove("Süt")
print("Süt çıkarıldıktan sonra:", alisveris_listesi)
# Çıktı: Süt çıkarıldıktan sonra: ['Elma', 'Ekmek', 'Yoğurt']

alisveris_listesi = "Kırmızı Elma" # İlk elemanı güncelle
print("Güncellenmiş alışveriş listesi:", alisveris_listesi)
# Çıktı: Güncellenmiş alışveriş listesi: ['Kırmızı Elma', 'Ekmek', 'Yoğurt']
```

Bu alıştırmada, öğrencilerden oluşan bir not listesi içerisindeki en yüksek ve en düşük notu bulacağız. Bunun için Python'un yerleşik `max()` ve `min()` fonksiyonlarını kullanacağız. Uygulama Adımları:

1. Önceden tanımlanmış bir not listesi oluştur.
2. `max()` fonksiyonunu kullanarak en yüksek notu bul.
3. `min()` fonksiyonunu kullanarak en düşük notu bul.
4. Ortalama notu hesapla (ekstra bir uygulama).

```
notlar =  
en_yuksek_not = max(notlar)  
en_dusuk_not = min(notlar)  
ortalama_not = sum(notlar) / len(notlar)  
  
print(f"Öğrenci Notları: {notlar}")  
print(f"En Yüksek Not: {en_yuksek_not}") # Çıktı: En Yüksek Not: 100  
print(f"En Düşük Not: {en_dusuk_not}")   # Çıktı: En Düşük Not: 65  
print(f"Ortalama Not: {ortalama_not:.2f}") # Çıktı: Ortalama Not: 85.38
```

Demet (Tuple), listeye oldukça benzer bir koleksiyon tipidir. Ancak önemli bir farkı vardır:

- Değiştirilemezdir (Immutable) : Demetler oluşturulduktan sonra elemanları üzerinde ekleme, silme veya güncelleme yapılamaz. Bu özellik, veri bütünlüğünün korunması gereken durumlarda onu değerli kılar.
- Sıralıdır : Elemanlar belirli bir sıraya göre saklanır ve indeks ile erişilebilir.
- Genellikle fonksiyonlardan birden fazla değer döndürülürken kullanılır.

Demetler, parantezler () arasına elemanlar yazılarak oluşturulur. Kod Örneği:

```
koordinat = (10, 25)
```

```
print(koordinat) # Çıktı: 10
```

```
print(koordinat[1]) # Çıktı: 25
```

```
bilgiler = ("Ahmet", "Yılmaz", 30)
```

```
print(bilgiler) # Çıktı: ('Ahmet', 'Yılmaz', 30)
```

Demetlerin değiştirilemez (immutable) olması, bazı durumlarda listelere göre avantaj sağlar:

- Veri Bütünlüğü : Bir kez tanımlandıktan sonra değişmemesi gereken veriler için idealdir (örneğin, bir noktanın koordinatları, bir kişinin doğum tarihi).
- Fonksiyon Dönüş Değerleri : Fonksiyonlardan birden fazla değer döndürülürken demetler sıklıkla kullanılır. Bu, değerlerin bir arada ve değişmez bir şekilde gruplanmasını sağlar.
- Sözlük Anahtarları : Değişmez oldukları için demetler, sözlüklerde anahtar olarak kullanılabilirken, listeler kullanılamaz.

Bir fonksiyonun birden fazla değeri bir demet olarak döndürmesi yaygın bir kullanım senaryosudur. Kod:

```
def daire_hesapla(yaricap):  
    pi = 3.14159  
    alan = pi * yaricap**2  
    cevre = 2 * pi * yaricap  
    return alan, cevre # Demet olarak döndürülür  
  
r = 5  
daire_alan, daire_cevre = daire_hesapla(r) # Değerler demetten ayrıştırılır  
print(f"Yarıçapı {r} olan dairenin alanı: {daire_alan:.2f}, çevresi:  
{daire_cevre:.2f}")  
# Çıktı: Yarıçapı 5 olan dairenin alanı: 78.54, çevresi: 31.42
```

Sözlük (Dictionary), her bir elemanın bir anahtar (key) ile bir değerin (value) eşleştirildiği bir yapıdır.

- Anahtar-Değer Yapısı : Her veri parçası, ona erişmek için kullanılan benzersiz bir anahtarla ilişkilendirilir.
- Hızlı Veri Erişimi : Anahtarlar üzerinden verilere çok hızlı bir şekilde erişim sağlar.
- Sırasız (Python 3.7+ ile sıralı hale geldi) : Eski Python versiyonlarında sıralı değildi. Python 3.7 ve sonrasında, ekleme sırası korunur.
- Değiştirilebilirdir (Mutable) : Oluşturulduktan sonra elemanlar eklenebilir, silinebilir veya güncellenebilir.
- JSON veri formatının temelini oluşturur, bu nedenle web uygulamalarında ve veri alışverişinde çok önemlidir.

Sözlükler, süslü parantezler {} arasına anahtar-değer çiftleri yazılarak oluşturulur. Elemanlara anahtarları kullanarak erişilir. Kod Örneği:

```
ogrenci = {  
    "ad": "Ali",  
    "soyad": "Can",  
    "numara": 101,  
    "not_ortalamasi": 87.5  
}  
  
print(ogrenci["ad"])          # Çıktı: Ali  
print(ogrenci["numara"])      # Çıktı: 101
```

Sözlüklere yeni bir anahtar-değer çifti eklemek veya mevcut bir anahtarın değerini güncellemek için basit atama işlemi kullanılır. Kod Örneği:

```
urun = {"isim": "Laptop", "fiyat": 15000}
```

```
urun["stok"] = 50 # Yeni eleman ekleme
```

```
print(urun) # Çıktı: {'isim': 'Laptop', 'fiyat': 15000, 'stok': 50}
```

```
urun["fiyat"] = 14500 # Mevcut elemanı güncelleme
```

```
print(urun) # Çıktı: {'isim': 'Laptop', 'fiyat': 14500, 'stok': 50}
```

Sözlükten anahtar-değer çiftlerini silmek için del anahtar kelimesi veya pop() metodu kullanılabilir.

- `del sozluk[anahtar]` : Belirtilen anahtara sahip öğeyi siler.
- `sozluk.pop(anahtar)` : Belirtilen anahtara sahip öğeyi siler ve silinen değeri döndürür. Kod Örneği:

```
personel = {"id": 1, "ad": "Ayşe", "departman": "İK", "maaş": 60000}
del personel["maaş"] # 'maaş' anahtarını sil
print(personel) # Çıktı: {'id': 1, 'ad': 'Ayşe', 'departman': 'İK'}
```

```
silinen_departman = personel.pop("departman") # 'departman'ı sil ve değerini al
print(personel) # Çıktı: {'id': 1, 'ad': 'Ayşe'}
print(f"Silinen departman: {silinen_departman}") # Çıktı: Silinen departman
```

Sözlük Metotları: keys(), values(), items()

Sözlüklerin anahtarlarına, değerlerine veya her ikisine birden erişmek için özel metotlar bulunur.

- `sozluk.keys()` : Sözlükteki tüm anahtarları içeren bir görünüm (view object) döndürür.
- `sozluk.values()` : Sözlükteki tüm değerleri içeren bir görünüm döndürür.
- `sozluk.items()` : Sözlükteki tüm anahtar-değer çiftlerini (tuple olarak) içeren bir görünüm döndürür. Kod Örneği:

```
ayarlar = {"tema": "koyu", "dil": "Türkçe", "bildirimler": True}
print(ayarlar.keys())    # Çıktı: dict_keys(['tema', 'dil', 'bildirimler'])
print(ayarlar.values())  # Çıktı: dict_values(['koyu', 'Türkçe', True])
print(ayarlar.items())   # Çıktı: dict_items([('tema', 'koyu'), ('dil', 'Türkçe'),
('bildirimler', True)])
```

```
for key, value in ayarlar.items():
    print(f"{key}: {value}")
```

Bir öğrencinin bilgilerini (ad, soyad, numara, notlar) bir sözlük yapısında saklamak, anahtar-değer yapısının gerçek dünya kullanımına iyi bir örnektir. Bu sayede bilgilere anahtarları üzerinden hızlıca erişebiliriz. Uygulama Adımları:

1. Öğrencinin bilgilerini içeren bir sözlük oluştur.
2. Anahtarları kullanarak bilgilere eriş.
3. Yeni bir bilgi ekle (örneğin, ders bilgisi).



Kod Örneği: Öğrenci Bilgilerini Saklama ve Erişim

```
ogrenci_bilgileri = {  
    "ad": "Ahmet",  
    "soyad": "Yılmaz",  
    "numara": 123,  
    "notlar": {"Matematik": 85, "Fizik": 70, "Kimya": 90}  
}  
  
print(f"Öğrenci Adı: {ogrenci_bilgileri['ad']}") # Çıktı: Öğrenci Adı: Ahmet  
print(f"Öğrenci Numarası: {ogrenci_bilgileri['numara']}") # Çıktı: Öğrenci Numarası: 123  
print(f"Matematik Notu: {ogrenci_bilgileri['notlar']['Matematik']}") # Çıktı: Matematik Notu: 85  
  
ogrenci_bilgileri["bolum"] = "Bilgisayar Mühendisliği" # Yeni bilgi ekle  
print(ogrenci_bilgileri)  
# Çıktı: {'ad': 'Ahmet', 'soyad': 'Yılmaz', 'numara': 123, 'notlar': {'Matematik': 85, 'Fizik': 70, 'Kimya': 90}, 'bolum': 'Bilgisayar Mühendisliği'}
```


Bu alıştırma, kullanıcıdan alınan bir metin içerisindeki her bir harfin kaç kez tekrar ettiğini hesaplayan ve sonucu bir sözlük olarak { 'a': 5, 'b': 2, ... } ekrana yazdıran bir program yazmayı içerir. Bu, döngülerin ve sözlüklerin birlikte kullanımını pekiştiren harika bir örnektir. Uygulama Adımları:

1. Kullanıcıdan bir metin girdisi al.
2. Metni küçük harflere çevirerek harf sayımını kolaylaştır.
3. Her harfin sayısını tutmak için boş bir sözlük oluştur.
4. Metin üzerindeki her karakteri döngü ile gez.
5. Eğer karakter bir harf ise, sözlükte sayısını artır.
6. Sonuç sözlüğü ekrana yazdır.

```
metin = input("Bir metin girin: ")  
# Kullanıcıdan "Merhaba Dünya!" girdisi alındığını varsayalım  
print(f"Girilen Metin: {metin}")
```



```
metin = "Merhaba Dünya!" # Önceki adımdan alınan metin
harf_frekanslari = {}

for karakter in metin.lower(): # Metni küçük harfe çevirip karakter karakter gez
    if 'a' <= karakter <= 'z' or karakter in "çğğlïoöşü": # Yalnızca harfleri say
        if karakter in harf_frekanslari:
            harf_frekanslari[karakter] += 1
        else:
            harf_frekanslari[karakter] = 1

print("Harf Frekansları Hesaplanıyor...")
```

```
# harf_frekanslari sözlüğü önceki adımdan gelir
print("Harf Frekansları:")
for harf, frekans in sorted(harf_frekanslari.items()):
    print(f"'{harf}': {frekans}")

# Çıktı (örnek metin: "Merhaba Dünya!"):
# 'a': 2
# 'b': 1
# 'd': 1
# 'e': 1
# 'h': 1
# 'm': 1
# 'n': 1
# 'r': 1
# 'ü': 1
# 'y': 1
```



Küme (Set), içerisinde yalnızca benzersiz (tekrarsız) elemanlar barındıran, sırasız bir koleksiyondur.

- Benzersiz Elemanlar : Bir küme aynı elemandan birden fazlasını içeremez. Otomatik olarak tekrarları eler.
- Sırasız : Elemanların belirli bir sırası yoktur ve indeksleme ile erişilemez.
- Değiştirilebilirdir (Mutable) : Kümeye eleman eklenebilir veya silinebilir, ancak içerisindeki elemanların kendileri değiştirilemez (elemanların immutable olması gerekir).
- Matematiksel küme işlemleri (kesişim, birleşim, fark) için idealdir.

Kümeler, süslü parantezler {} arasında elemanlar yazılarak oluşturulur. Boş küme oluşturmak için set() fonksiyonu kullanılır. Kod Örneği:

```
sayi_kumesi = {1, 2, 3, 2, 1}
print(sayi_kumesi) # Çıktı: {1, 2, 3} (Tekrarlar otomatik elendi)
```

```
harfler = set("abracadabra")
print(harfler) # Çıktı: {'r', 'a', 'b', 'c', 'd'} (Sıra garanti değil, benzersiz)
```

```
bos_kume = set()
bos_kume.add(10)
print(bos_kume) # Çıktı: {10}
```



Kümeler, matematiksel küme işlemlerini kolayca yapmamızı sağlar.

- Birleşim (`union()` veya `|`) : İki kümenin tüm farklı elemanlarını içeren yeni bir küme oluşturur.
- Kesişim (`intersection()` veya `&`) : İki kümenin ortak elemanlarını içeren yeni bir küme oluşturur.
- Fark (`difference()` veya `-`) : Bir kümede olup diğerinde olmayan elemanları içeren yeni bir küme oluşturur. Kod Örneği:

```
kume1 = {1, 2, 3, 4}
kume2 = {3, 4, 5, 6}
birlesim = kume1.union(kume2) # veya kume1 | kume2
print(f"Birleşim: {birlesim}") # Çıktı: Birleşim: {1, 2, 3, 4, 5, 6}
kesisim = kume1.intersection(kume2) # veya kume1 & kume2
print(f"Kesişim: {kesisim}") # Çıktı: Kesişim: {3, 4}
fark = kume1.difference(kume2) # veya kume1 - kume2
print(f"Fark (kume1 - kume2): {fark}") # Çıktı: Fark (kume1 - kume2): {1, 2}
```



Neden Küme (Set) Kullanılır?

Kümelerin benzersizlik ve sırasızlık özellikleri, onları belirli görevler için çok kullanışlı hale getirir.

- Tekrarlayan Elemanları Eleme : Bir listedeki yinelenen elemanları hızlıca tekilleştirmek için kullanılır. Bir listeyi kümeye dönüştürüp tekrar listeye çevirerek bu işlem yapılabilir.
- Üyelik Testi : Bir elemanın bir koleksiyon içinde olup olmadığını hızlıca kontrol etmek için (in operatörü) kümeler çok etkilidir.
- Matematiksel Küme İşlemleri : Ortak elemanları, farklı elemanları veya tüm elemanları bulmak gerektiğinde (örneğin, iki grubun ortak hobileri).
- Büyük veri kümelerinden benzersiz değerleri çıkarmak gibi durumlarda performans avantajı sağlar.

String (Metin), karakter dizileridir ve Python'da temel bir veri tipidir. Genellikle tek veya çift tırnak arasına yazılırlar.

- Değiştirilemezdir (Immutable) : Bir string oluşturulduktan sonra içeriği değiştirilemez. Eğer bir stringi değiştirmeye çalışırsanız, aslında yeni bir string oluşturulur.
- Sıralıdır ve indeksleme ile karakterlere erişilebilir.
- Metin verileri üzerinde birçok faydalı işlem ve metot bulunur.



String Metotları: .upper() ve .lower()

Metin verileri üzerinde yaygın olarak kullanılan .upper() ve .lower() metotları, metnin büyük veya küçük harflere dönüştürülmesini sağlar.

- string.upper() : Metnin tüm karakterlerini büyük harfe dönüştürür.
- string.lower() : Metnin tüm karakterlerini küçük harfe dönüştürür. Kod Örneği:

```
metin = "Python Programlama"
```

```
buyuk_metin = metin.upper()
```

```
kucuk_metin = metin.lower()
```

```
print(f"Orijinal: {metin}")          # Çıktı: Orijinal: Python Programlama
```

```
print(f"Büyük Harf: {buyuk_metin}") # Çıktı: Büyük Harf: PYTHON PROGRAMLAMA
```

```
print(f"Küçük Harf: {kucuk_metin}") # Çıktı: Küçük Harf: python programlama
```

String Metotları: .split()

`string.split(ayraç)` metodu, bir metni belirli bir ayraç karakterine göre bölerek bir listeye dönüştürür. Ayraç belirtilmezse, varsayılan olarak boşluk karakterine göre böler ve birden fazla boşluğu tek bir ayraç gibi ele alır. Kod Örneği:

```
cumle = "Merhaba dünya, nasılsın?"
kelimeler = cumle.split(" ")
print(f"Kelimeler: {kelimeler}") # Çıktı: Kelimeler: ['Merhaba', 'dünya,', 'nasılsın?']

veri = "elma,armut,çilek"
meyveler = veri.split(",")
print(f"Meyveler: {meyveler}") # Çıktı: Meyveler: ['elma', 'armut', 'çilek']
```

String Metotları: .join()

`ayraç.join(liste)` metodu, bir liste veya demet gibi yinelenabilir (iterable) bir koleksiyonun elemanlarını, belirtilen ayraç stringini kullanarak tek bir stringde birleştirir.

- `split()` metodunun tersi olarak düşünülebilir. Kod Örneği:

```
kelimeler_listesi = ["Bu", "bir", "örnek", "cümledir."]
```

```
birlesik_cumle_bosluk = " ".join(kelimeler_listesi)
```

```
print(f"Boşlukla birleşmiş: {birlesik_cumle_bosluk}") # Çıktı: Boşlukla birleşmiş: Bu bir örnek cümledir.
```

```
sayilar_str = ["1", "2", "3", "4"]
```

```
birlesik_sayilar_tire = "-".join(sayilar_str)
```

```
print(f"Tireyle birleşmiş: {birlesik_sayilar_tire}") # Çıktı: Tireyle birleşmiş: 1-2-3-4
```

Bu alıştırmada, kullanıcıdan alınan bir cümle için `split(' ')` metodu ile kelimelerine ayrılıp bir listeye dönüştürülmesi ve ardından `len()` fonksiyonu ile cümledeki kelime sayısının bulunması gösterilecektir. Uygulama Adımları:

1. Kullanıcıdan bir cümle girdisi al.
2. `split()` metodunu kullanarak cümleyi kelimelere ayır ve bir liste oluştur.
3. `len()` fonksiyonunu kullanarak bu kelime listesinin uzunluğunu bul.
4. Sonucu ekrana yazdır.

```
girdi_cumle = input("Bir cümle girin: ")  
# Kullanıcıdan "Python öğrenmek çok eğlenceli." girdisi alındığını varsayalım  
  
kelimeler = girdi_cumle.split() # Parametresiz split, birden fazla boşluğu tek  
boşluk olarak sayar  
kelime_sayisi = len(kelimeler)  
  
print(f"Girilen cümle: '{girdi_cumle}'")  
print(f"Cümledeki kelime sayısı: {kelime_sayisi}")  
# Çıktı:  
# Girilen cümle: 'Python öğrenmek çok eğlenceli.'  
# Cümledeki kelime sayısı: 4
```

Her veri yapısının belirli senaryolarda diğerlerine göre daha avantajlı olduğu durumlar vardır.

- Liste (List) :

- Sıralı bir koleksiyona ihtiyacınız olduğunda.
- Elemanları sık sık eklemeniz, silmeniz veya güncellemeniz gerektiğinde.
- Farklı veri tiplerini bir arada saklamak istediğinizde.
- Örnek: Bir alışveriş listesi, bir film koleksiyonu.



- Demet (Tuple) :

- Elemanların oluşturulduktan sonra değiştirilmemesi gereken durumlarda (veri bütünlüğü).
- Fonksiyonlardan birden fazla değeri tek bir birim olarak döndürmek istediğinizde.
- Sözlük anahtarı olarak kullanılabilecek sıralı veri gruplarına ihtiyacınız olduğunda.
- Örnek: Bir noktanın (x, y) koordinatları, bir kişinin (ad, soyad, yaş) bilgileri.



◦ Sözlük (Dictionary) :

- Verilere anahtar-değer çiftleri olarak erişmeniz gerektiğinde.
- Anahtarlar üzerinden çok hızlı veri erişimi önemli olduğunda.
- Yapılandırılmış verileri (örneğin, JSON) temsil etmek istediğinizde.
- Örnek: Bir öğrencinin detaylı bilgileri, ürünlerin fiyat listesi.



◦ Küme (Set) :

- Yalnızca benzersiz (tekrarsız) elemanlar saklamak istediğinizde.
- Sıranın önemli olmadığı durumlarda.
- Matematiksel küme işlemleri (birleşim, kesişim, fark) yapmanız gerektiğinde.
- Bir koleksiyondaki yinelenen elemanları temizlemek istediğinizde.
- Örnek: Bir sınıftaki tüm benzersiz öğrenci ID'leri, iki farklı grubun ortak üyeleri.

Bu ders haftasında, Python'daki temel koleksiyon tipi veri yapılarını (Liste, Demet, Sözlük, Küme) tanıdık.

- Her birinin kendine özgü avantajlarını ve kullanım senaryolarını öğrendik.
- Listeler ve sözlükler üzerinde temel manipülasyon işlemlerini (ekleme, silme, güncelleme, erişim) kod örnekleriyle uyguladık.
- String verileri üzerinde yaygın metotları kullanarak metin işleme becerilerimizi geliştirdik. Veri yapılarının doğru ve etkin kullanımı, daha verimli ve anlaşılır kodlar yazmanın temelini oluşturur.

Çalışma Soruları

Hafta 2 – Veri Yapıları ve Manipülasyon

Çalışma Soruları

1. Veri Yapıları Karşılaştırması: Liste, Demet, Sözlük ve Küme veri yapılarını aşağıdaki üç kritere göre karşılaştırınız:

- Değiştirilebilirlik (Mutable/Immutable)
- Sıralama (Ordered/Unordered)
- Eleman Tekrarı (Benzersizlik)

2. Senaryo Bazlı Seçim: Aşağıdaki senaryoların her biri için en uygun veri yapısının hangisi olduğunu belirtiniz ve nedenini açıklayınız:

- Bir öğrencinin sınav notlarını (birden çok not olabilir) saklamak.
- Bir fonksiyonun, bir hesaplama sonucunda hem sonucu hem de hata kodunu birlikte döndürmesi.
- Kullanıcıdan alınan bir metindeki her bir harfin kaç kez geçtiğini saymak.
- Bir anketin "Hangi sosyal medya platformlarını kullanıyorsunuz?" sorusuna verilen tüm benzersiz cevapları toplamak.

3. Demet'in Avantajları: Bir Demet (Tuple) veri yapısının "değiştirilemez" (immutable) olmasının sağladığı temel avantajlar nelerdir? Fonksiyonlardan değer döndürme senaryosunda neden Demetler sıkça tercih edilir?

4. Sözlük ve JSON ilişkisi: Sözlük (Dictionary) veri yapısının, web uygulamaları ve veri alışverişinde sıkça kullanılan JSON formatının temelini oluşturduğu belirtilmiştir. Bu ilişkiyi açıklayınız. Sizce bu benzerlik neden önemlidir?

5. Liste Manipülasyonu: Bir liste üzerinde gerçekleştirilebilen temel manipülasyon işlemlerini (ekleme, silme, güncelleme, dilimleme) açıklayınız. `append()` ve `remove()` metotlarının işlevlerini birer örnekle gösteriniz.

6. String Metotları: `.split()` ve `.join()` metotlarının işlevlerini açıklayınız. Bu iki metodun birbirinin tersi olarak kabul edilebileceğini bir kod örneği üzerinden gösteriniz.

Deneme Soruları

Hafta 2 – Veri Yapıları ve Manipülasyon



Deneme Soruları

1. Bir listenin belirli bir bölümünü almak için kullanılan işleme ne ad verilir? A) Bölme (Splitting) B) Birleştirme (Joining) C) Dilimleme (Slicing) D) Güncelleme (Updating)
2. Aşağıdaki veri yapılarından hangisi elemanları oluşturulduktan sonra değiştirilemez (immutable)? A) Liste B) Demet C) Sözlük D) Küme
3. Bir öğrencinin adı, soyadı ve numarası gibi bilgileri bir arada tutmak için anahtarlar üzerinden hızlı erişim sağlayan en uygun veri yapısı hangisidir? A) Liste B) Demet C) Küme D) Sözlük
4. `metin = "Veri Yapıları"` komutundan sonra `metin.lower()` komutu çalıştırıldığında `metin` değişkeninin son değeri ne olur? A) "VERI YAPILARI" B) "veri yapıları" C) "Veri Yapıları" D) Hata verir
5. İçerisinde yalnızca benzersiz (tekrarsız) elemanlar barındıran ve matematiksel küme işlemleri için ideal olan veri yapısı hangisidir? A) Liste B) Demet C) Küme D) Sözlük
6. `cümle = "Bu bir deneme cümlesidir"` komutundan sonra `kelimeler = cümle.split(' ')` komutu çalıştırıldığında `kelimeler` değişkeninin tipi ne olur? A) String B) Liste C) Demet D) Sözlük
7. `notlar = [85, 92, 78]` listesine `notlar.append(100)` komutu uygulandıktan sonra listenin son hali ne olur? A) `[100, 85, 92, 78]` B) `[85, 92, 78, 100]` C) `[85, 100, 92, 78]` D) `[85, 92, 78]`
8. Bir metin içerisindeki her bir harfin kaç kez tekrar ettiğini hesaplayan "Harf Frekansı Hesaplayıcı" uygulamasında, harfleri ve tekrar sayılarını saklamak için hangi veri yapısının kullanılması önerilmiştir? A) Liste B) Demet C) Sözlük D) Küme
9. `ogrenci = {'ad': 'Ahmet', 'soyad': 'Yılmaz'}` sözlüğünden 'Ahmet' değerine erişmek için hangi ifade kullanılır? A) `ogrenci[0]` B) `ogrenci('ad')` C) `ogrenci.get('Ahmet')` D) `ogrenci['ad']`
10. Bir not listesi içerisindeki en yüksek notu bulmak için kullanılan yerleşik Python fonksiyonu hangisidir? A) `high()` B) `max()` C) `top()` D) `upper()`



ÖZET



Süleyman **EZDEMİR**

suleyman.ezdemir@giresun.edu.tr

